

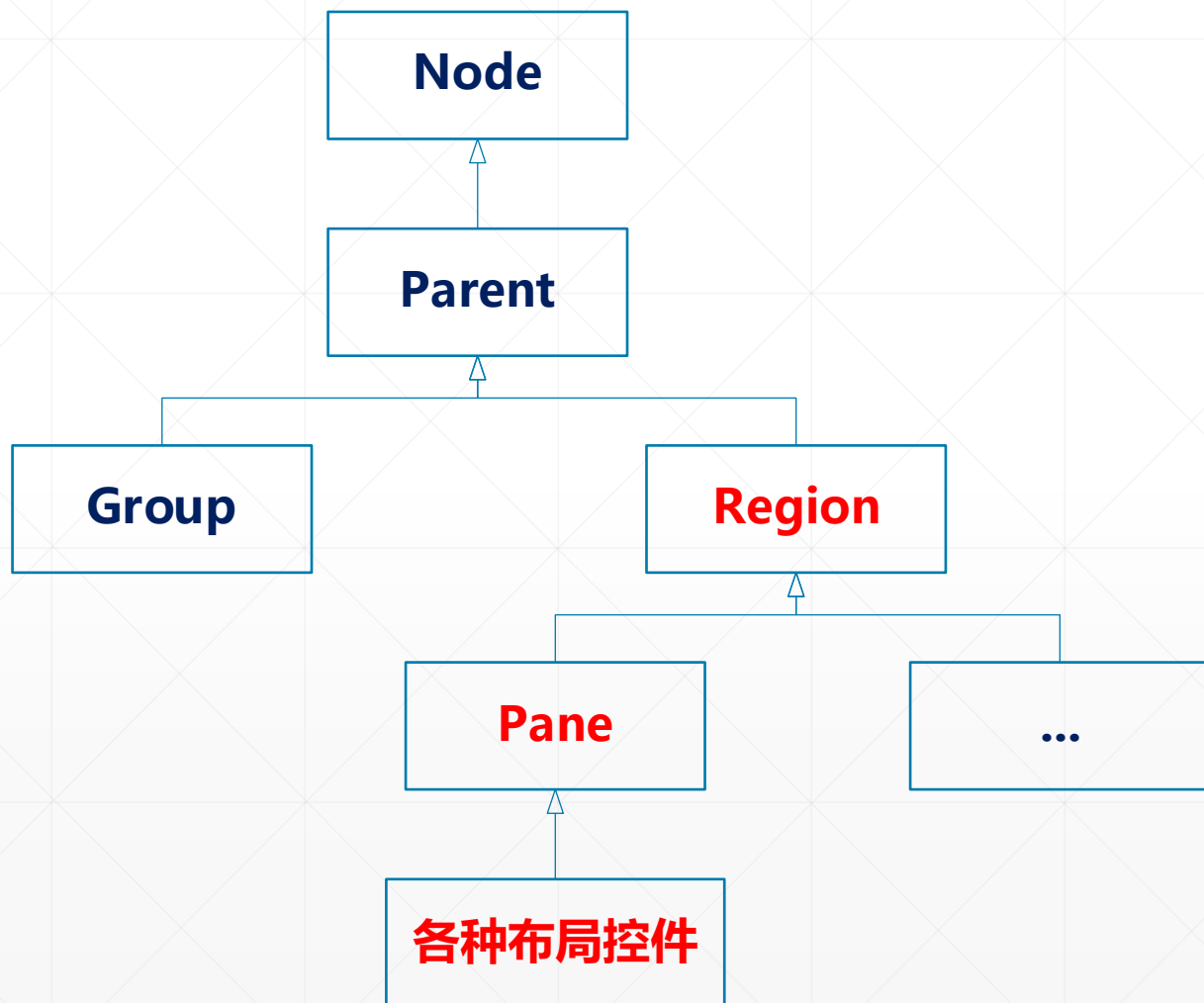
JavaFX常用控件之



布局控件的基类型

北京理工大学计算机学院
金旭亮

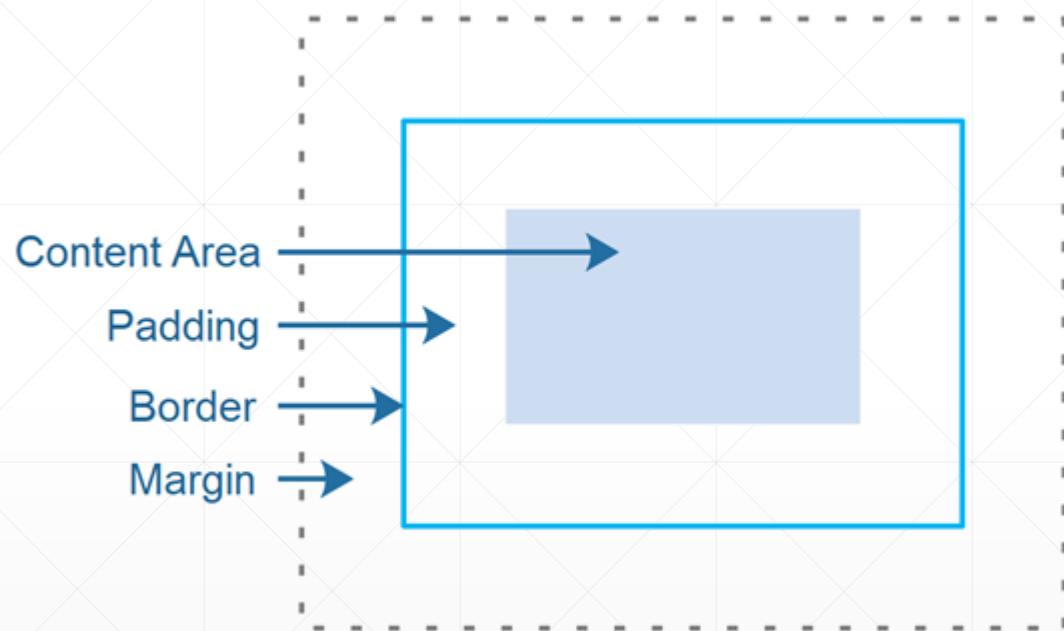
JavaFX中布局控件的类型树



日常使用的各种布局控件，大多数都是**Pane**的子类，而**Pane**又是**Region**和**Parent**的子类，因此，了解这几个基类型的相关情况，对于学习与掌握特定的布局控件来说，是很有必要的。

Parent是所有有子控件的控件的基类，主要提供那些公用的功能，比如添加与移除子控件，触发界面重绘等，一般情况下，不需要太多关注它。但它的两个子类比较重要，**Group**另外介绍，先说说**Region**.....

Region类型简介



Region是所有需要布局的控件的基类，提供了对CSS3的直接支持。

Region最重要的地方，就是它定义了所有JavaFX控件的“布局”属性和“背景”属性。

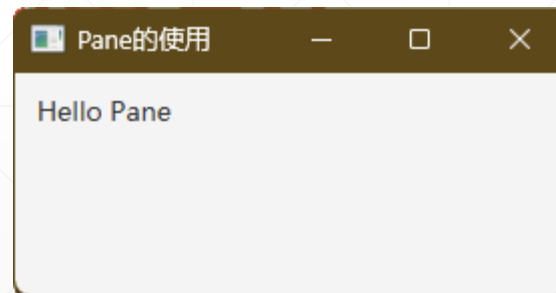
Region同时支持使用代码和CSS两种方式定制这些属性值。

Pane1概述



各种布局控件，都是Pane类的子类。Pane公开了子控件的集合，所以，布局控件都可以动态地添加和移除子控件。

```
Label lblInfo = new Label( text: "Hello Pane");  
lblInfo.setPadding(new Insets( topRightBottomLeft: 10));  
  
Pane pane = new Pane();  
pane.getChildren().add(lblInfo);
```



获取Children集合的引用，再调用add/addAll()方法将控件添加到布局控件中，是通用编程模式。

Pane控件的排列与样式设置

Pane自己不负责排列控件，默认情况下，它在左上角显示它的所有子控件，因此，程序员往往需要自己写代码来定位特定的控件。

```
Button okBtn = new Button( s: "OK");
Button cancelBtn = new Button( s: "Cancel");
okBtn.relocate( x: 10, y: 10);
cancelBtn.relocate( x: 60, y: 10);

Pane root = new Pane();
root.getChildren().addAll(okBtn, cancelBtn);
root.setStyle("-fx-border-style: solid inside;" +
              "-fx-border-width: 3;" +
              "-fx-border-color: red;");
```



Pane支持使用CSS调整显示样式

设置Pane的背景



有两种方式设置Pane的背景，一种是使用CSS样式，另一种是使用对象。

使用CSS设置背景比较简洁，推荐采用。

```
//使用样式设置Pane的背景色
1 usage
public Pane getCSSStyledPane() {
    Pane p = new Pane();
    p.setPrefSize( prefWidth: 100, prefHeight: 50);
    p.setStyle("-fx-background-color: lightgray, red;"
        + "-fx-background-insets: 0, 4;"
        + "-fx-background-radius: 4, 2;");
    return p;
}
```

使用代码设置Pane的背景



1 usage

```
public Pane getObjectStyledPane() {  
    Pane p = new Pane();  
    p.setPrefSize( prefWidth: 100, prefHeight: 50);  
  
    BackgroundFill lightGrayFill =  
        new BackgroundFill(Color.LIGHTGRAY,  
            new CornerRadii( radius: 4), new Insets( topRightBottomLeft: 0));  
  
    BackgroundFill redFill =  
        new BackgroundFill(Color.RED,  
            new CornerRadii( radius: 2), new Insets( topRightBottomLeft: 4));  
  
    // 使用两个BackgroundFill对象创建一个Background对象  
    Background bg = new Background(lightGrayFill, redFill);  
    p.setBackground(bg);  
  
    return p;  
}
```

Pane的背景，使用BackgroundFill对象定义的颜色进行填充，因此，必须先实例化一个BackgroundFill对象，再基于它实例化Background对象，之后才能调用setBackground()方法设定Pane的背景，比较麻烦，还是推荐使用CSS样式设置背景。

Pane可作为“画布（Canvas）”

Pane的子控件，可以通过设置它的左上角坐标实现绝对定位，这个特点，让它成为一个绝佳的“绘图面板”，可以在它上面排列各种图形（Shape）对象。

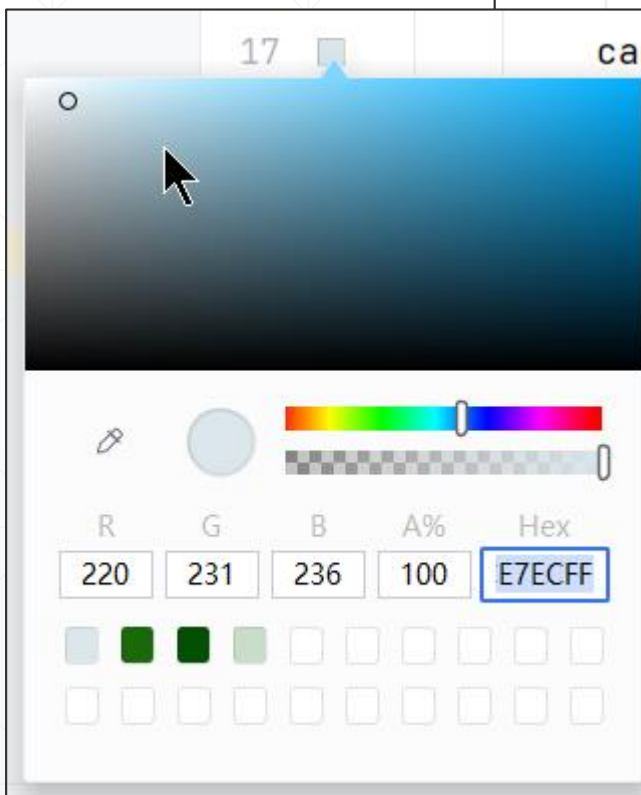


Pane最适合的场景，是基于Shape实现“自由”绘图。

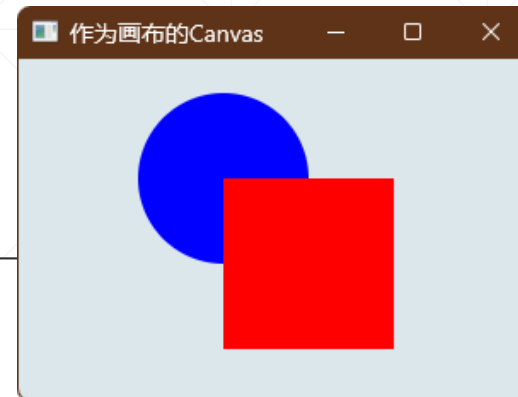
使用Canvas绘图

小技巧

点击

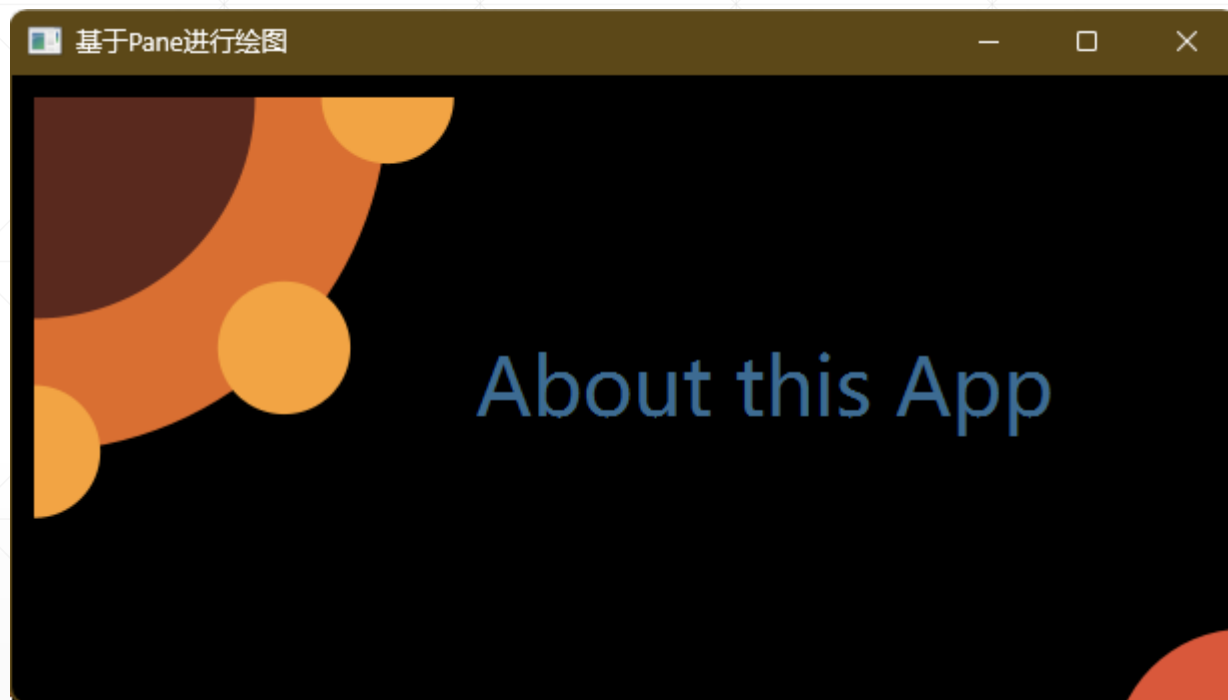


```
Pane canvas = new Pane();  
//使用样式设置Pane背景是最方便的方法  
canvas.setStyle("-fx-background-color: #dce7ec;");  
//设置绘图尺寸  
canvas.setPrefSize( prefWidth: 300, prefHeight: 200);  
//添加两个图形: 圆和矩形, 并且为其定位  
Circle circle = new Circle( radius: 50, Color.BLUE);  
circle.relocate( x: 70, y: 20);  
Rectangle rectangle = new Rectangle( width: 100, height: 100, Color.RED);  
rectangle.relocate( x: 120, y: 70);  
//将所有图形, 添加到面板中  
canvas.getChildren().addAll(circle, rectangle);
```



IntelliJ IDEA能识别出JavaFX代码中的样式, 点击代码左侧的小方块, 可以调出颜色设置面板。

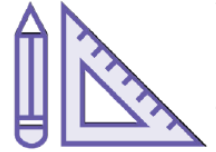
Pane“绘图”复杂示例



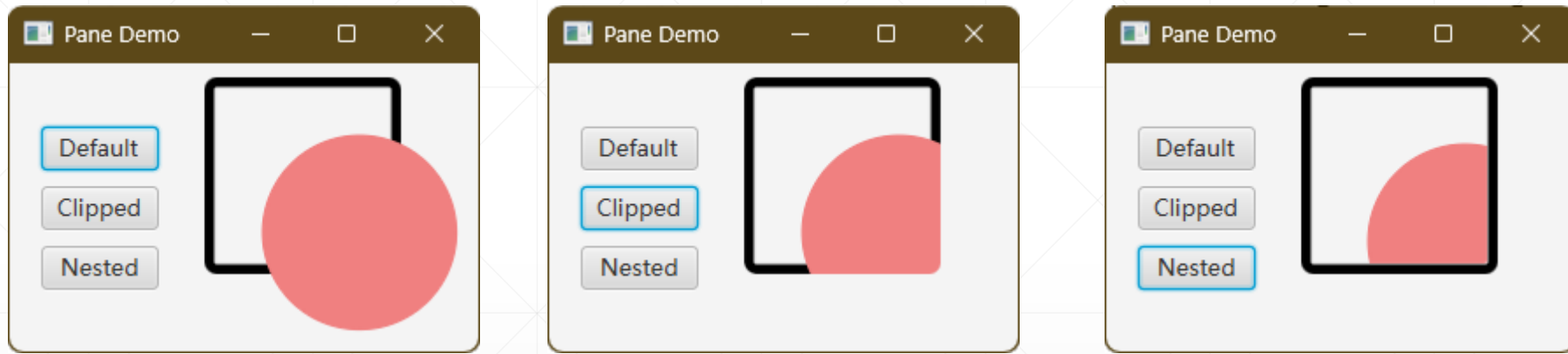
这个示例，其实就是在Pane上面用JavaFX的基本形状“堆”出一幅图。

基本思路就是创建好相应的几何图形和文本对象，事先计算并设置好它们的大小和显示位置，然后，全部加入到Pane的children集合中。

图形剪裁



当Pane所包容的图形对象超过了Pane的边界时，Pane可以剪裁这些图形，如以下示例所示：



这个示例很清晰地展示出来了Clip的作用与效果。由于在实际开发中，这个特性用得不多，同学们对此有所了解就行。